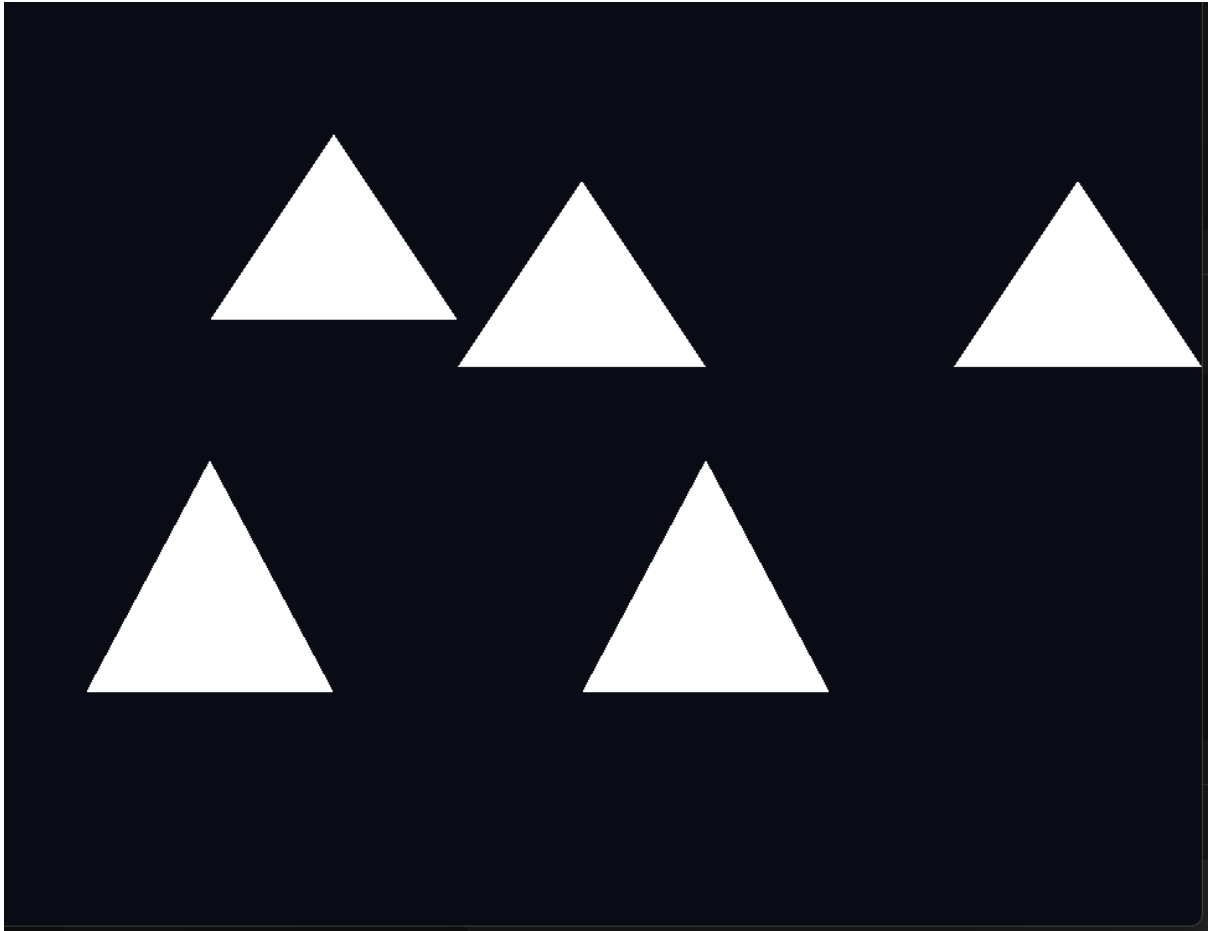


Report Visual Computing Fundamentals

Yiming Hou

31/08/2025

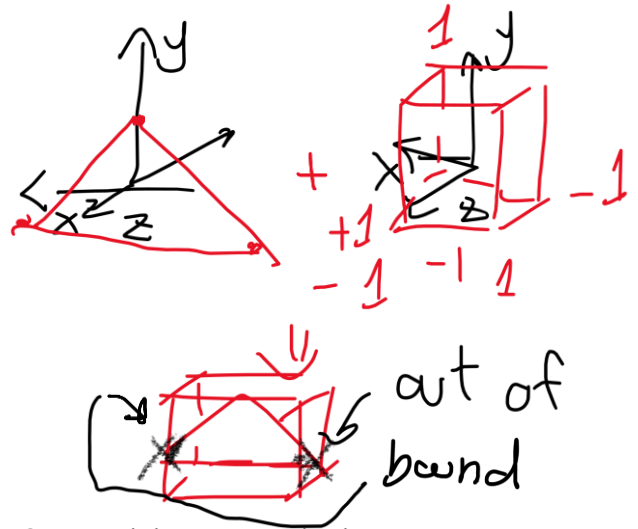
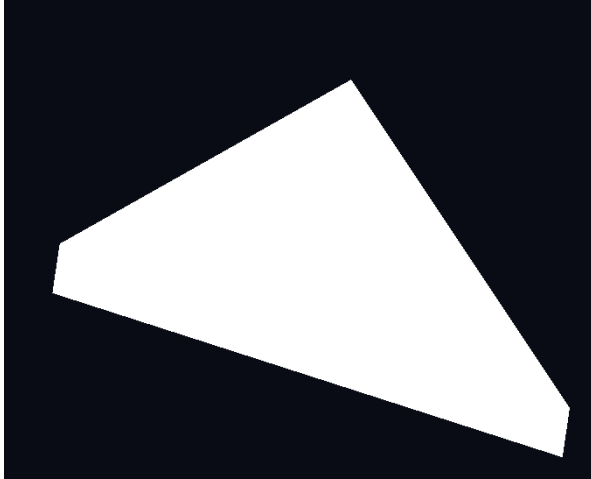
Task 1



5 random triangles in the 2D plane (xy)

Task 2

a)



In OpenGL the visible space inside the screen is called Clipbox, it ranges from -1 to +1 in all three directions. Among the coordinates provided there were values bigger than 1 and smaller than -1 which makes them not fit inside the field of view and therefore causes the “distortion” since part of the triangle is not within the field of view. In other words this phenomenon is due to the size of the triangle compared to the field of view (**Clipping**)

Furthermore since the coordinates given are three dimensional (notice that in the first task all the values of the coordinate z were set at zero) the different points of the triangle will have different depth. The result is a projection in a 2D space we are seeing.

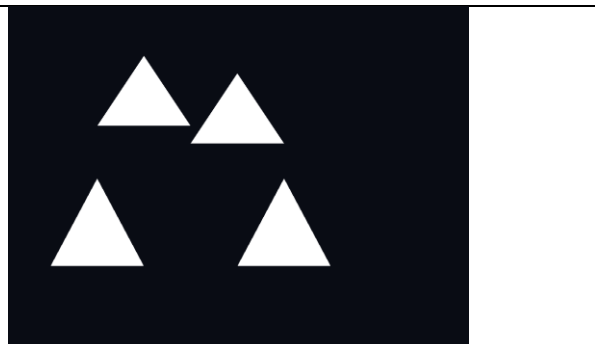
(**Perspective distortion**)

In OpenGL the GPU rasterizes the primitives as triangles. Its purpose is to let OpenGL rasterize any geometric shape and triangles are the simplest planar polygon which let the rendering be fast.

```
let vertices: Vec<f32> = vec![
    // task2 q1
    0.6, -0.8, -1.2,
    0.0, 0.4, 0.0,
    -0.8, -0.2, 1.2
];
// Triangle 1 // -> rand
```

b)

After swapping the order of the vertex of the fourth triangle (far right)



Before

```
// Triangle 3
-0.2, 0.2, 0.0, // left
0.2, 0.2, 0.0, // right
0.0, 0.6, 0.0, // top

// Triangle 4
0.6, 0.2, 0.0, // left
1.0, 0.2, 0.0, // right
0.8, 0.6, 0.0, // top

// Triangle 5
-0.6, 0.3, 0.0, // left
-0.2, 0.3, 0.0, // right
-0.4, 0.7, 0.0, // top*/
];

let indices: Vec<u32> = vec![
0, 1, 2, // Triangle 1
3, 4, 5, // Triangle 2
6, 7, 8, // Triangle 3
9, 10, 11, // Triangle 4
12, 13, 14, // Triangle 5*/
];
```

After

```
// Triangle 4
0.6, 0.2, 0.0, // left
1.0, 0.2, 0.0, // right
0.8, 0.6, 0.0, // top

// Triangle 5
-0.6, 0.3, 0.0, // left
-0.2, 0.3, 0.0, // right
-0.4, 0.7, 0.0, // top*/
];

let indices: Vec<u32> = vec![
0, 1, 2, // Triangle 1
3, 4, 5, // Triangle 2
6, 7, 8, // Triangle 3
9, 11, 10, // Triangle 4
12, 13, 14, // Triangle 5*/
];
```

- i) The triangle with the swapped vertices is not visible anymore
- ii) Because the triangle is not seen from the front: in fact by swapping the order, the rotation to draw the triangle become clockwise, which is the negative rotation, and the triangle is therefore seen from the back and not visible
- iii) The order of the indices must be counterclockwise which is the positive sign for the rotation. This is to let the object being viewed from the front, so it will be visible for the observer

c)

- i. The depth buffer keeps track of the distance between the viewer and the point, in other word it holds the “depth” of the point. Once OpenGL renders the composition, if some points happen to have the same value on the plane xy but different z, only the part with the lowest z coordinate value gets rendered because the points with higher z coordinates are not seen as there is something else blocking the field of view before them. Sometimes there might be an effect called Z fighting when the points have a negligible difference of the z coordinate value among them, which causes effects such as overlapping planes. When the scene changes not only the frame buffer must be cleared but also the depth buffer because otherwise all the new pixels will be compared against the

depth values from the previous frame with outdated information that must be discarded.

If the depth buffer isn't cleared, a moving sphere would appear to leave a “trail” or “ghosts”.

- ii. Assuming no usage of multisampling every fragment is executed once, including fragments that will be discarded later due depth test, therefore if two or more fragments happen to be overlapped on the same pixel the Fragment Shader will be executed for each overlapping fragment on the same pixel although only the one closest to the viewer will be shown after the depth testing and will have its colour actually written in the frame buffer.
- iii. The two most commonly used types of Shaders- programs are for vertices and for fragments. In OpenGL these shaders are written in the OpenGL Shading Language (GLSL) and are essential components of the rendering pipeline.
The vertex shader processes individual vertices of a 3D model and it transform vertices from local space to the clip space, and send these data further down the pipeline
The fragment shader determines the colour of each fragment that makes up the image. It's executed for every fragment, and its main role is to calculate the final colour of a pixel, taking into account factors like lighting, texturing, and material properties. It receives the interpolated values from the vertex shader as input.
- iv. An index buffer is used to avoid repeating the same vertices that belong to multiple triangles because otherwise there will be duplicate vertices in the VBO. Storing each vertex once, and then referencing them by index saves memory, avoids redundancy and allows GPU vertex processing to be reused for shared vertices.
- v. The last parameter is an offset, so if the data you need is right at the beginning there is no need to insert an offset but if multiple attributes are stored in the same buffer (interleaved data), then certain types of data won't be at the very beginning of the buffer so an offset is needed to reach where that attribute's data begins.

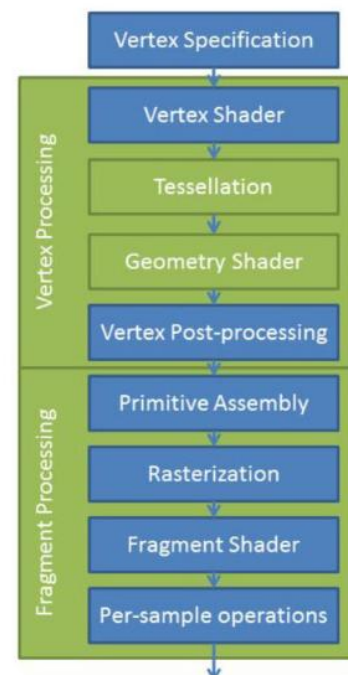


Fig 1 from slides of Visual computing fundamentals course at NTNU

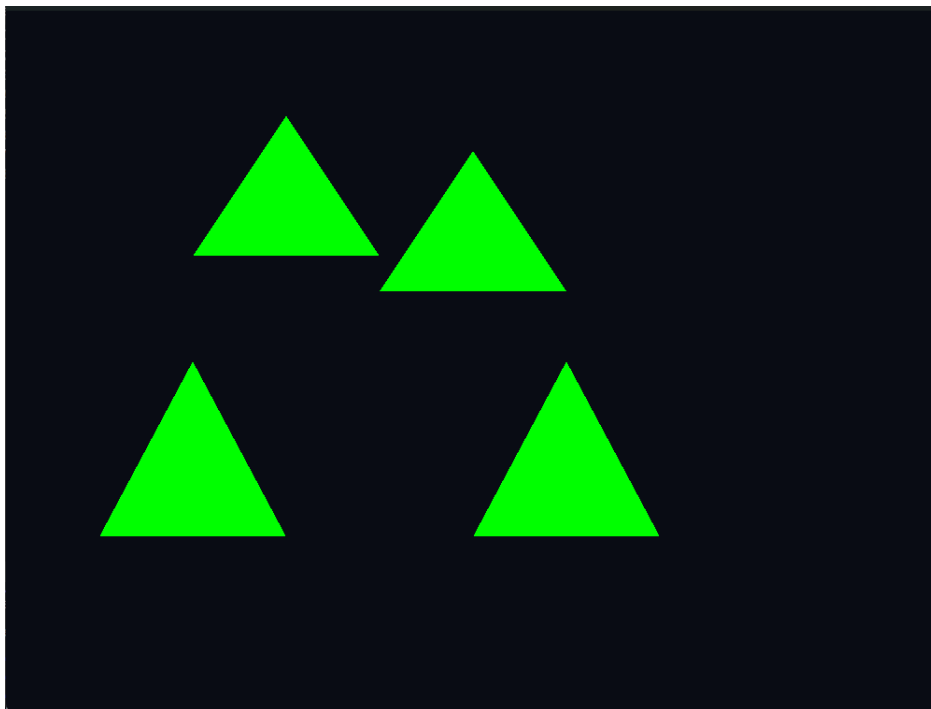
d)

i) To flip both horizontally and vertically I have multiplied x and y coordinates by -1 inside simple.vert which mirrored the scene on both axis



ii) I have modified the values given to the function `vec4()` which gives the colour to the image inside `simple.frag` (the fragment shader) to change the colour from white to green.

I have done task ii) first and then proceeded with task i)



Optional bonus challenge

e) a single square formed by two triangles

